

Практика 1: Проектирование API и взаимодействия

Table of contents

Практика 1: Проектирование API и взаимодействия	1
Предисловие	1
Задание	2

Практика 1: Проектирование API и взаимодействия

Предисловие

В лекциях рассматриваются технические плюсы и минусы различных архитектурных подходов: базовые определения и “здравый смысл” в выборе паттернов — когда их стоит применять, а когда нет.

В этой практике принцип простой: сначала проектируем, потом кодим. Цель — выбрать осмысленный бизнес-сценарий по варианту и заранее спроектировать API, слои и сценарии приложения.

Почему в практиках мы отталкиваемся от Clean Architecture

Строгая архитектура даёт два ключевых свойства: **единообразный путь данных и общий словарь**.

Единообразный путь данных означает, что любой запрос проходит одну и ту же цепочку слоёв. Без этого каждый новый endpoint изобретает собственную структуру — особенно заметно при работе с llm:

Промпт	Результат
Добавь endpoint для получения пользователя по ID	<code>GET /users/:idhandler → UserRepository → ORM</code>
Добавь endpoint для получения списка задач	<code>GET /taskshandler → ORM (напрямую)</code>
Добавь endpoint для получения статистики пользователя	<code>GET /users/:id/statshandler → UserRepository → StatsManager → ORM</code>

Три endpoint'а — три разных решения. При отладке непонятно, где искать логику. С ростом кодовой базы хаос усугубляется.

Общий словарь (`entity`, `use case`, `adapter`, `repository`) решает смежную проблему: в больших командах одни и те же роли называют `service`, `observer`, `manager`. Единые термины убирают лишний “перевод” между командами и упрощают ревью и онбординг.

При этом важно не превращать терминологию в догму. Паттерны вроде Strategy, Facade, Observer никуда не исчезают — они живут внутри конкретных компонентов. Строгая архитектура фиксирует внешние границы и путь данных, а не запрещает локальные решения.

Задание

1. Выберите продукт и сценарий работы.

В реальной разработке бизнес описывает путь пользователя: последовательность шагов, которые человек проходит для достижения цели. Именно из этих шагов рождаются endpoints, сценарии приложения и слои системы. Опишите сценарий пользователя на 2-3 шага бизнес-логики (не CRUD).

Примеры сценариев по вариантам:

- **Интернет-магазин:** пользователь открывает каталог → выбирает категорию → получает список товаров с фильтрами (цена, рейтинг) → добавляет товар в корзину → система проверяет наличие на складе → если товара нет, предлагает аналоги → оформляет заказ → система резервирует товар и создаёт заказ со статусом “ожидает оплаты”
- **Сервис доставки еды:** пользователь указывает адрес → система показывает рестораны, которые доставляют в эту зону → пользователь выбирает ресторан и блюда → система считает сумму с доставкой → пользователь подтверждает → система ищет свободного курьера и назначает его
- **Такси/каршеринг:** пользователь указывает точку А и Б → система рассчитывает маршрут и стоимость по тарифу → пользователь подтверждает → система ищет ближайшего свободного водителя → назначает водителя → отправляет пользователю ETA
- **Образовательная платформа:** студент открывает курс → видит модули и свой прогресс → открывает урок → проходит тест → система проверяет ответы → если пройден порог, открывает следующий модуль → если все модули пройдены, генерирует сертификат
- **Банковское приложение:** пользователь выбирает “перевод” → вводит номер счёта получателя → система проверяет существование счёта → вводит сумму → система проверяет достаточность средств → списывает с одного счёта, зачисляет на другой → создаёт две записи в истории транзакций
- **Медицинская система:** пациент выбирает специальность врача → система показывает врачей с ближайшими свободными слотами → пациент выбирает время → система проверяет, что слот не занят → создаёт запись → отправляет подтверждение

- **Система бронирования билетов:** пользователь выбирает событие → видит схему зала с доступными местами → выбирает места → система временно резервирует их (5 минут) → пользователь подтверждает → система переводит бронь в “оплачен” и генерирует билет
- **Социальная сеть:** пользователь открывает ленту → система собирает посты от подписок, сортирует по времени → пользователь ставит лайк → счётчик обновляется → пишет комментарий → автор поста получает уведомление

В каждом сценарии есть проверки, условия, несколько сущностей, которые меняют состояние вместе. Именно это и нужно спроектировать.

Внешние сервисы. Некоторые шаги сценария требуют знаний, которых пока нет (геопоиск ближайшего водителя, генерация PDF, отправка email, рекомендации). Это нормально — разрешается сказать: “эту операцию выполняет отдельный внешний сервис” и описать его контракт. Например: `NearestDriverService.Find(location, radius)` → `Driver` или `NotificationService.Send(userId, message)`. В Практике 2 этот контракт реализуется заглушкой (возвращает фиксированный результат), а в дальнейших практиках, возможно, будет реализован по-настоящему. Ограничение: основной сценарий должен жить в вашем приложении, наружу можно вынести 1-2 операции.

2. Выберите интерфейс и протокол.

- Варианты: REST (OpenAPI), GraphQL, gRPC, fullstack (HTML endpoints), комбинации.
- Менее стандартные подходы:
 - **Fullstack (сервер отдаёт HTML):** Phoenix, Django, htmx
 - **Внутренние микросервисы:** gRPC-вызов к отдельному сервису, который выполняет тяжёлую работу
- Кратко обоснуйте выбор. Примеры обоснований:
 - **Система бронирования билетов → REST.** Мы продаём API партнёрам (агрегаторы, виджеты на сайтах площадок). Клиенты — внешние, на разных языках и платформах. REST + OpenAPI spec даёт им документацию и кодогенерацию клиента из коробки.
 - **Банковский сервис переводов → gRPC.** Сервис работает внутри периметра компании, его вызывают другие микросервисы (мобильный BFF, процессинг). Важна типобезопасность контрактов (.proto) и скорость бинарной сериализации. Внешних потребителей нет.
 - **Интернет-магазин → fullstack (HTML).** Страницы каталога и карточки товаров должны индексироваться поисковиками (SEO). SSR решает проблему — сервер отдаёт готовый HTML, поисковой бот видит контент сразу.
 - **Мобильное приложение соцсети → GraphQL.** Экраны сильно различаются: лента показывает пост + автор + счётчик лайков, профиль — аватар + подписчики + последние посты. REST потребовал бы десятки endpoint’ов или возвращал бы лишние поля. GraphQL позволяет каждому экрану запросить ровно те данные, которые нужны.

- Если не хочется тратить время на выбор — REST всегда безопасный вариант. В этом и ценность Clean Architecture: бизнес-логика живёт в сценариях приложения и не зависит от интерфейса. Если позже окажется, что нужен gRPC или GraphQL — меняется только слой адаптеров, а domain и сценарии приложения остаются без изменений.

3. Определитесь со стеком.

- Выберите язык, веб-фреймворк и подход к DI. Код в этой практике писать не нужно — но к Практике 2 вы должны чётко понимать, на чём будете реализовывать.

Популярные экосистемы для ориентира:

Язык	Веб-фреймворк	DI	Сильные стороны
Go	chi, echo, fiberoapi-codegen (OpenAPI yml first)нативный gRPC	wire (кодогенерация)fx (Uber)ручной DI в main.go	gRPC из коробки, OpenAPI yml first, sqlc для типобезопасного SQL
Python	FastAPI (OpenAPI из коробки)Django REST Framework-Flask	FastAPI (встроен)dependency-injector	Быстрый старт, автодокументация API в FastAPI
Java / Kotlin	Spring BootQuarkus, MicronautKtor	Spring (встроен)Dagger 2 (код-генерация)Koin (Kotlin)	IoC Зрелая экосистема, Spring-манифест всё
C#	ASP.NET Core Minimal APIASP.NET Core MVC	встроен в ASP.NET CoreAutofac	DI из коробки, сильная типизация
TypeScript	Express, FastifyNestJS (встроен)Next.js, bun	NestJS (встроен)tsyringe, InversifyJS	Fullstack на одном языке, Next.js для SSR
Elixir	Phoenix (LiveView / JSON API)	behaviours + конфигурация	Полный стек с поддержкой реального времени из коробки

4. Опишите API-контракты.

- Для каждого endpoint/query/mutation: входные данные, выходные данные, ошибки.
- Формат: OpenAPI yml, [.proto](#), GraphQL schema или структурированная таблица.

5. Опишите архитектуру приложения.

- Слои, компоненты, направление зависимостей, интерфейсы между слоями.
- На схеме **обязательно** явно различайте:
 - реализационные компоненты (класс/модуль/сервис),
 - интерфейсные компоненты (порт/контракт).
- Для зависимостей укажите тип стрелки или подпись связи:
 - `depends on` (компонент зависит от интерфейса),
 - `implements` (компонент реализует интерфейс).
- По умолчанию ожидается Clean Architecture. Допустимы другие подходы (MVC, framework-based и т.д.) — при условии, что вы аргументируете выбор и так же подробно расписываете слои, зависимости и контракты между ними.

Пример эталонной схемы (Clean Architecture):

На схеме показан усложнённый, но типовой сценарий: несколько входных протоколов и несколько внешних хранилищ/сервисов. Стрелки направлены к центральному слою сценариев приложения и отражают правило зависимостей Clean Architecture (зависимости направлены внутрь). Для этой работы минимальное требование: не менее 4 компонентов в каждом слое — входные обработчики, сценарии и порты. Требования по авторизации в этой практике отсутствуют: фокус на архитектуре и бизнес-логике.

Что сдаётся:

1. Краткое описание продукта и выбранного сценария.
2. Обоснование выбора протокола.
3. Выбор стека (язык, фреймворк, DI).
4. Спецификация API (endpoint/query/mutation + request/response/errors).
5. Схема слоёв, контрактов и зависимостей с явным различием интерфейсных и реализационных компонентов.

Теория к сдаче:

Лекция 1 — Принципы построения приложения:

- Что такое компонент? Контракт (интерфейс) компонента. Примеры компонентов разного масштаба (функция, класс, сервис)
- Что такое зависимость между компонентами? Почему неконтролируемые зависимости — проблема?
- Что такое слой? Правила межслойных зависимостей
- Продуктивность слоя: прокси-компоненты, коэффициент проксирования, когда слой не нужен
- Принципы SOLID (уметь объяснить каждый на примере)
- Dependency Injection: зачем нужен, как реализуется
- Clean Architecture: слои (domain, use cases, adapters, infrastructure), правило зависимостей (зависимости направлены внутрь)

Лекция 2 — Альтернативные архитектурные паттерны:

- Framework-based архитектура (Rails/Django): что значит “opinionated framework”, плюсы и минусы
- Модель акторов (Erlang/Akka): изолированные процессы, обмен сообщениями, “Let it Crash”, когда применяется
- Data-Oriented Design: зачем оптимизировать под кеш процессора, где применяется

Лекция 3 — Протоколы: REST / gRPC:

- REST: HTTP методы, коды ответов, JSON, URI endpoints, OpenAPI/Swagger
- gRPC: Protocol Buffers, HTTP/2, кодогенерация, стриминг
- Когда REST, когда gRPC? (публичный API vs внутренние сервисы)

Лекция 4 — Протоколы: GraphQL / WebSocket:

- GraphQL: проблема over-fetching/under-fetching, единственный endpoint, клиент описывает структуру ответа
- WebSocket: проблема polling, постоянное двустороннее соединение, применение (чаты, работа в реальном времени)

Лекция 5 — Асинхронные взаимодействия: брокеры, очереди, обмен файлами:

- Зачем асинхронность: развязка сервисов, устойчивость, сглаживание пиков нагрузки
- Брокеры сообщений и очереди: Kafka/RabbitMQ, pub/sub vs очереди задач, at-least-once/at-most-once
- Событийное взаимодействие: события домена, eventual consistency, саги
- Надежность: идемпотентность, дедупликация, ретраи, DLQ
- Обмен файлами/шаринг: когда уместно, плюсы и риски, контроль версий и целостности